AI-BASED PLAGIARISM DETECTION & CODE SIMILARITY ANALYZER

PROJECT REPORT

1. Abstract

The AI-Based Plagiarism Detection & Code Similarity Analyzer is a software project designed to compare source-code submissions and identify potentially similar code. Users provide two code samples through a web interface, and the backend processes the input and returns a similarity result according to the comparison method implemented in the application. The tool can support code review and academic integrity checks. A similarity score should be treated as an indicator for human review, not as proof of plagiarism.

2. Problem Statement

Manually comparing multiple programming assignments can be time-consuming. A simple application that accepts code samples and displays a comparison result can help reviewers identify submissions that may need closer inspection.

3. Objectives

- Provide a clear interface for entering or submitting code samples.
- Connect the frontend to a backend analysis service.
- Compare two inputs using the algorithm implemented in the project.
- Display the result in an understandable form.
- Maintain project code and documentation in GitHub.

4. Technologies Used

Frontend: HTML, CSS and JavaScript (verify against the source files). Backend: Python and Flask if used in the implementation. Version control: Git and GitHub. Development and local testing: Visual Studio Code and a web browser.

5. System Design and Workflow

The system follows a frontend-backend workflow. The user opens the website and supplies two code samples. The frontend validates the input and sends it to the backend API. The backend processes the samples using the implemented similarity logic and returns a response. The frontend then displays the similarity score or an error message.

User → Frontend → Backend API → Similarity Analysis → Result

6. Main Modules

Frontend Module: presents input controls, action buttons, validation messages and the result area. Backend Module: receives requests, validates data, performs analysis and returns a response. Similarity Module: compares the submitted code according to the algorithm present in the project.

7. Testing Plan

Test case	Input / Action	Expected result
Identical code	Submit the same code twice.	High or maximum similarity, depending on the algorithm.
Different code	Submit unrelated code samples.	Generally lower similarity than identical inputs.
Empty input	Leave one or both inputs empty.	Validation message or controlled error.
Backend unavailable	Stop backend and analyze.	Clear connection error, not a silent failure.
Repeat test	Run the same comparison again.	Consistent results for deterministic logic.

8. How to Run the Project

1. Open the Code-Similarity-Analyzer repository in Visual Studio Code.
2. Install the dependencies listed in backend/requirements.txt, if that file exists.
3. Start the backend using the run command specified by the project. If backend/app.py is a runnable Flask app, python backend/app.py may be suitable.
4. Open frontend/index.html with Live Server if the frontend is a static HTML application.
5. Confirm that the API address in the frontend matches the backend address, then test with sample code.

9. Expected Outcome

When configured correctly, the application should accept valid code samples, send them to the backend, and show the returned similarity result. It should also handle empty inputs and backend connection failures with understandable messages. Exact features and supported languages depend on the current implementation.

11. Conclusion

The Code Similarity Analyzer combines a web interface with backend processing to make source-code comparison easier. It provides a starting point for identifying potentially similar submissions and supports learning about frontend-backend integration, testing and version control. Results should be reviewed by a person and should not be used alone to make a plagiarism determination.

Repository

https://github.com/6606137-svg/Code-Similarity-Analyzer

Screenshots to Add

Add actual screenshots of the homepage, code comparison form, and similarity result to the screenshots/ folder. Only include features that have been implemented and tested.